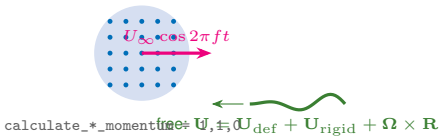
CONSTRAINT IB

Rigid and prescribed-motion bodies with `ConstraintIBMethod`: a kinematics class, the input blocks, the force files, and three upstream examples run



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 14

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say what the constraint IB method does that the standard IB method cannot: make a region of fluid move as a rigid body, or as a body whose deformation you prescribe, by correcting the fluid velocity inside it after every solve rather than by springs and beams;
2. read a ConstraintIB program: `ConstraintIBMethod` in place of `IBMethod`, a kinematics class that returns the velocity you want, and the two input blocks that tie them to a structure;
3. write the `ConstraintIBKinematics` block for a prescribed body, a free rigid body and a free swimmer, with the keys that exist (the 2020 slides had one that does not);
4. find the forces, velocities and positions the method writes, and read them with four lines of Python or MATLAB;
5. build and run the three examples of this chapter (an oscillating cylinder, an impulsively started cylinder, a swimming eel) and say what each shows.

> Chapter 13 gave a point mass; this chapter gives a whole body a rigid motion, prescribed or free. Where the standard IB method asks “what force does this shape produce?”, the constraint method asks “what force does it take to make the fluid move like this?” and applies it.

HOW THIS CHAPTER WAS CHECKED

- Every input key, error message and output file is quoted from the IBAMR 0.19.0 sources (`ConstraintIBMethod.cpp`, `ConstraintIBKinematics.cpp`, the three examples' kinematics classes); the step-by-step account of slide 17 is the method's `postprocessSolveFluidEquations`, in order.
- The three examples ConstraintIB/01, 02 and 03 are the upstream `oscillating_rigid_cylinder`, `impulsively_started_cylinder` and `eel2d`, each with its edits listed in its README; they build with 0 warnings against 0.19.0, their MATLAB generators were run under Octave 8.4 and the Python ports write the same files byte for byte (and the same files as shipped upstream), and the figures are from the runs made for this chapter.
- The 2020 lecture *Looking at ConstraintIB examples of prescribed and rigid body motion* (IBAMR Lecture 7, 23 slides) and the six ConstraintIB slides of Lecture 6 are quoted in full; where a slide was wrong about the code (a key that does not exist, a file name, what one class does), the slide is quoted and the correction follows it.

> Slide numbers refer to this deck; chapter numbers (5, 6, 7, 10, 11, 13, 15, 16, 26) to the other chapters of this series.

WHERE TO LOOK THINGS UP

- The method: `include/ibamr/ConstraintIBMethod.h`, `src/IB/ConstraintIBMethod.cpp` (the constructor opens the output files; `getFromInput` lists the keys; `postprocessSolveFluidEquations` is the algorithm).
- The kinematics interface: `include/ibamr/ConstraintIBKinematics.h` (four pure virtual functions) and `src/IB/ConstraintIBKinematics.cpp` (the `StructureParameters` constructor reads the structure block and names the three update methods).
- The examples: `examples/ConstraintIB/` in the IBAMR source tree, nine folders (slide 7); this series ships three of them as ConstraintIB/01–03 with READMEs and measured results.
- The paper: A. P. S. Bhalla, R. Bale, B. E. Griffith and N. A. Patankar, *A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies*, J. Comput. Phys. 250 (2013) 446–476. Section numbers on slide 7 are its.
- Related tools: `CIBMethod` (chapter 15), `IBFEDirectForcingKinematics`, `BrinkmanPenalizationRigidBodyDynamics` (slide 35).

WHAT THE CONSTRAINT IB METHOD DOES

The standard IB method (chapters 5–13) computes a force from the structure's deformation and lets the fluid carry the structure. The constraint method turns this round:

1. the fluid is solved everywhere, inside the body too, as if the body were fluid;
2. the new fluid velocity is interpolated to the body's points, $\mathbf{U}$;
3. the velocity the body *should* have is assembled,

$$\mathbf{U}_{\mathrm{new}} = \mathbf{U}_{\mathrm{def}} + \mathbf{U}_{\mathrm{rigid}} - \mathbf{U}_{\mathrm{def,COM}} + \boldsymbol{\Omega} \times \mathbf{R},$$

   from the prescribed deformation velocity and the rigid translation and rotation, which are either prescribed (locked at zero) or computed from the momentum of the fluid in the body;
4. the correction $(\mathbf{U}_{\mathrm{new}} - \mathbf{U})$ is spread back to the grid, so the fluid in the body region moves as the body does; the force this took, $\rho_s(\mathbf{U}_{\mathrm{new}} - \mathbf{U})/\Delta t$ per point, is the constraint force, and minus its sum is the hydrodynamic force on the body.

No springs, no beams, no stiffness to choose and no stiffness-limited time step: the body is rigid to the accuracy of the interpolation and the fluid solver's CFL limit is the only one. The price: the body is a region of fluid of the fluid's density ($\rho_{\mathrm{solid}} = \rho_{\mathrm{fluid}}$ is assumed; a heavier body gets a body force instead, slide 18), and a deforming body must have its deformation prescribed, not computed from elasticity (that is IBFE's job, Part IV).

THE 2020 LECTURE'S FRAMING

IBAMR Lecture 7 (2020), slide 2, repeating Lecture 6's slide 18: "Constraint IB Examples. *A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies*, Amneet Pal Singh Bhalla, Rahul Bale, Boyce E. Griffith, Neelesh A. Patankar. This paper is on the google drive. Note that it is the first paper for rigid body simulations. Nearly all of the examples considered in this paper appear in these Constraint IB examples. Some examples are also similar to the CIB ones. Some of the examples generate HUGE numbers of files."

Slide 3 was a screenshot of the folder `examples/ConstraintIB` with its nine sub-folders (slide 7). Lecture 6, which toured IBAMR's example tree, had introduced four of them in one slide each; Lecture 7 then opened three of them file by file: the oscillating cylinder (prescribed motion), the impulsively started cylinder (rigid-body motion) and the 2D eel (prescribed deformation, free swimming). This chapter follows that order and runs all three.

> "HUGE numbers of files": the upstream inputs dump visualization every 40 steps and, in the eel's case, restart files every 150 steps, for runs of 100,000 steps and more. The examples of this series shorten the runs and the dump intervals (each README's "History").

THE NINE UPSTREAM EXAMPLES (1): PRESCRIBED MOTION

folder	body and motion	paper	the lecture (2020)
`stokes_first_problem`	a plate started impulsively	7.1.1	“an infinitely long horizontal plate is impulsively started with a velocity U_0 in the x direction”
`moving_plate`	a finite plate dragged across the fluid	7.1.2	“a finite plate of height b in an infinite quiescent flow that is dragged perpendicular to itself with a constant velocity U_0”
`oscillating_rigid_cylinder`	a disc oscillating; ConstraintIB/01	7.1.4	“contributions to the drag forces come from both inertial and constraint forces”
`flow_past_cylinder`	a disc towed at constant speed	–	(not shown; the same files as 01 with `prescribed_trans_velocity`)

All four lock every rigid degree of freedom (`calculate_*_momentum = 0,0,0`): the body moves as its kinematics class says and the force files are the result. Lecture 6 introduced the plate, the Stokes plate and the oscillating cylinder in one slide each (slides 20, 22, 23), quoting the paper’s section for each; Lecture 7 opened the oscillating cylinder file by file (slides 4–12). The paper’s quotes above are the lecture’s; its slide on the impulsively started cylinder read “D ¼ 1 m” and “16 mat time” where the PDF’s glyphs had been copied (it means $D = 1$ m and “16 m at time”).

THE NINE UPSTREAM EXAMPLES (2): FREE MOTION

folder	body and motion	paper	the lecture (2020)
`impulsively_started_cylinder`	a disc given an impulse; free rigid body; ConstraintIB/02	7.2.2	"to demonstrate the momentum-conservation properties of our method . . . an impulse of the form $I(t) = 400\exp(-400t)$ N/m^3 is applied in the x direction within the body"
`falling_sphere`	a sphere sinking under gravity, 3D; free	–	"we impose a gravitational body force that is present only in the solid region"
`eel2d`	a deforming eel, free swimming; ConstraintIB/03	7.3	"the body deforms in a prescribed manner but is propelled through the water due to rigid body motion"
`eel3d`, `knifefish`	the same in 3D; a knifefish with a ribbon fin	7.3	"It is worth running the 3D eel and knifefish as well."

These unlock translation and rotation (`1,1,0` and `0,0,1` in 2D; all six in 3D): the body takes the momentum of the fluid in it. Lecture 6, slide 24: "Other examples include free swimming cases of eel swimming in 2D and 3D as well as a 3D knifefish. I have run the 2D eel, but the download is taking too long to show today. It is worth running the 3D eel and knifefish as well. There is also a falling sphere example of rigid body motion in 3D. 'We validate our algorithm for a freely moving body by considering the problem of a sedimenting sphere. To treat this problem with our present formulation and implementation, which assume that the mass density of the fluid–structure system is uniform, we impose a gravitational body force that is present only in the solid region Ω_c of the physical domain Ω.' " Lecture 7 opened the impulsively started cylinder (slides 13–18) and the eel (19–23).

A CONSTRAINTIB PROGRAM: THE THREE PIECES

```
// main.cpp (ConstraintIB/01): 1. the method replaces IBMethod
#include <ibamr/ConstraintIBMethod.h>
const int num_structures =
    input_db->getIntegerWithDefault("num_structures", 1);
Pointer<ConstraintIBMethod> ib_method_ops = new ConstraintIBMethod(
    "ConstraintIBMethod",
    app_initializer->getComponentDatabase("ConstraintIBMethod"),
    num_structures);
Pointer<IBHierarchyIntegrator> time_integrator =
    new IBExplicitHierarchyIntegrator("IBHierarchyIntegrator",
        app_initializer->getComponentDatabase("IBHierarchyIntegrator
  "),
        ib_method_ops, navier_stokes_integrator);
time_integrator->initializePatchHierarchy(
    patch_hierarchy, gridding_algorithm);
// 2. one kinematics object per structure, from its sub-block of the
   input
vector<Pointer<ConstraintIBKinematics>> ibkinematics_ops_vec;
Pointer<ConstraintIBKinematics> ib_kinematics_op;
ib_kinematics_op = new OscillatingCylinderKinematics(
    "Cylinder",
    app_initializer->getComponentDatabase("ConstraintIBKinematics")
                   ->getDatabase("Cylinder"),
    ib_method_ops->getLDataManager(), patch_hierarchy);
ibkinematics_ops_vec.push_back(ib_kinematics_op);
// 3. register them, then build the method's operators and data
ib_method_ops->registerConstraintIBKinematics(
    ibkinematics_ops_vec);
ib_method_ops->initializeHierarchyOperatorsandData();
```

The rest of `main.cpp` is the program of IB/01 (chapter 6): the Navier–Stokes integrator, the grid, `IBStandardInitializer` reading the `.vertex` file, the boundary conditions, the viz writers and the time loop are unchanged. Three things are new:

1. `ConstraintIBMethod` takes the place of `IBMethod`, with the number of structures it will constrain;
2. after the hierarchy exists, one `ConstraintIBKinematics` object is made per structure, from the sub-block of `ConstraintIBKinematics` that bears the structure's name;
3. `initializeHierarchyOperatorsandData()` is called once the objects are registered.

A ConstraintIB program (2): the lecture's slides, and the other two examples' versions

The lecture's slides 11, 12 and 18 ("Example.cpp", "Example.cpp continued", "Other relevant parts of example.cpp") showed exactly the lines of slide 9: the `#include`, the `new ConstraintIBMethod(...)` with its three arguments, and the block that creates, registers and initializes the kinematics objects, for the oscillating cylinder (slide 12) and for the impulsively started one (slide 18).

Three things to notice:

- `num_structures`: the input may set it (02 and 03 do, `num_structures = 1`); the number of kinematics objects pushed into the vector must match it;
- the first argument of the kinematics constructor is an object name for restarts; the second is the *sub-block* of `ConstraintIBKinematics` bearing the structure's block name, so a renamed block needs a renamed `getDatabase`;
- `patch_hierarchy` is passed so that a class can look at the grid (the eel's does, to place its points one cell apart).

```
// ConstraintIB/02: a rigid body with no deformation
ib_kinematics_op = new RigidBodyKinematics(
    "CYLINDER_2D",
    app_initializer->getComponentDatabase("
  ConstraintIBKinematics")
                    ->getDatabase("Cylinder2d"),
    ib_method_ops->getLDataManager(), patch_hierarchy);
// ConstraintIB/03: the eel, plus the control-volume force
  evaluator
ib_kinematics_op = new IBEELKinematics(
    "eel2d",
    app_initializer->getComponentDatabase("
  ConstraintIBKinematics")
                    ->getDatabase("eel2d"),
    ib_method_ops->getLDataManager(), patch_hierarchy);
Pointer<IBHydrodynamicForceEvaluator> hydro_force =
    new IBHydrodynamicForceEvaluator("IBHydrodynamicForce",
        rho_fluid, mu_fluid, start_time, true);
hydro_force->registerStructure(box_X_lower, box_X_upper,
    patch_hierarchy, box_init_vel, 0);
```

The upstream `moving_plate` goes one step further: its `RigidBodyKinematics` object gets a callback, `registerRigidBodyKinematics(&COMTransVelocity, nullptr)`, a C function returning the plate's velocity, instead of input expressions.

THE KINEMATICS CLASS: FOUR FUNCTIONS, AND THE OSCILLATING CYLINDER'S

`ConstraintIBKinematics` is abstract; a subclass must provide

```
virtual void setKinematicsVelocity(const double time,
    const std::vector<double>&
  incremented_angle_from_reference_axis,
    const std::vector<double>& center_of_mass,
    const std::vector<double>& tagged_pt_position) = 0;
virtual const std::vector<std::vector<double>>&
  getKinematicsVelocity(const int level) const = 0;
virtual void setShape(const double time,
    const std::vector<double>&
  incremented_angle_from_reference_axis) = 0;
virtual const std::vector<std::vector<double>>& getShape(const
  int level) const = 0;
```

The method calls `setKinematicsVelocity` and then `setShape` once per step with the time, the body's accumulated rotation, its centre of mass and the position of the tagged point; it then reads back the velocity, `[component][point]` per level, and the shape (used only in the `CONSTRAINT_POSITION` modes). The base class's `StructureParameters` tells the subclass its index range, levels and which rigid components are free.

```
// OscillatingCylinderKinematics.cpp (upstream, unchanged)
if (input_db->keyExists("frequency")) d_freq = input_db->
  getDouble("frequency");
else TBOX_ERROR("... frequency of cylinder does not exist in
  the InputDatabase ...");
...
void OscillatingCylinderKinematics::
  setOscillatingCylinderSpecificVelocity(const double time)
{
    ...
    for (int k = 0; k < number_lag_points; ++k)
    {
        d_current_kinematics_vel[0][0][k] =
  d_prescribed_trans_vel + d_Uinf * std::cos(2 * M_PI * d_freq
  * time);
    }
}
```

"`d_current_kinematics_vel[0][0][k]` – this refers to level 0 (should usually be 0), the component of velocity (0 is x), and k is the vertex number." (2020, slide 6). The first index is the level *offset* from the structure's coarsest level, which is why it is 0 for a structure on one level; the class refuses a structure on more than one ("Total levels > 1").

THE CONSTRAINTIBKINEMATICS BLOCK: THE KEYS THAT EXIST

```
// input2d (ConstraintIB/01)
ConstraintIBKinematics {
     Cylinder {
     structure_names                  = "cylinder2d"
     structure_levels                 = MAX_LEVELS - 1
     calculate_translational_momentum = 0,0,0
     calculate_rotational_momentum    = 0,0,0
     lag_position_update_method       = "CONSTRAINT_VELOCITY
  "
     tagged_pt_identifier             = MAX_LEVELS - 1, 0
  // level, relative idx of lag point
     // Cylinder oscillates with U_infinity * cos(2*pi*freq*
  t)
     frequency                        = 0.2
     U_infinity                       = 1.0
     // Additional prescribed velocity
     prescribed_trans_velocity        = 0.0
    }
}
```

The sub-block's name (`Cylinder`) is what `main.cpp` asks for; the structure's name (`cylinder2d`) is the `.vertex` file's and must also be in `IBStandardInitializer`'s list, on the same level.

Read by the base class (`StructureParameters`), all required:

- `structure_names`, `structure_levels`: the structure(s) this object drives and their levels; a name not found gives "`Structure X does not exist on level N`";
- `calculate_translational_momentum`, `calculate_rotational_momentum`: three integers each, 1 for every rigid component that is *free* (computed from the fluid's momentum), 0 for locked; the third is the z direction, which 2D does not have;
- `lag_position_update_method`: `CONSTRAINT_VELOCITY` (positions follow the corrected velocity), `CONSTRAINT_POSITION` (positions set from the shape the class returns, plus the rigid motion) or `CONSTRAINT_EXPT_POSITION`; anything else stops with a message naming the three;
- `tagged_pt_identifier`: a level and a point index relative to the structure's first; the point's position is handed to `setKinematicsVelocity` every step (the eel uses it to steer).

The subclass's own keys: next slide.

THE CONSTRAINTIBKINEMATICS BLOCK (2): THE SUBCLASS'S OWN KEYS

Each kinematics class reads what it needs from the same sub-block, after the base class has read the six keys of slide 12. The three classes of this chapter:

class (example)	keys	if missing
`OscillatingCylinderKinematics` (01)	`frequency`, `U_infinity`; `prescribed_trans_velocity` (a steady drift added to the oscillation)	the first two: `TBOX_ERROR`, the run stops; the third defaults to 0
`RigidBodyKinematics` (02)	`kinematics_velocity_function_0`, `_1`: muParser expressions of `t` and of the centre of mass `X0`, `X1`, the velocity every point gets besides the rigid motion	a `TBOX_WARNING` per missing one and `"0.0"`
`IBEELKinematics` (03)	`body_shape_equation`; `deformation_velocity_function_0`, `_1` (expressions of `X_0`, `T` and the normal `N_0`, `N_1`); `initial_angle_body_axis_0`; `body_is_maneuvering`, `maneuvering_axis_equation`, `maneuvering_axis_is_changing_shape`; `food_location_in_domain_0`, `_1`	the shape equation and the food location are required; the velocity functions warn and default to `"0.0"`; the angle and the manoeuvring switches default to 0 and `FALSE` (the axis equation is read only when manoeuvring)

The parser variables: `t`/`T` for time, `X0`/`X_0`/`x0`/`x_0` for the first coordinate (the centre of mass in 02, the body coordinate in 03), and in the eel `N0`/`N_0` for the body normal; `pi`, `Pi`, `PI` are defined. These are the classes' conventions, not the method's: a class of your own reads whatever keys it declares.

THE CONSTRAINTIBMETHOD BLOCK AND PRINTOUTPUT

```
// input2d (ConstraintIB/01)
ConstraintIBMethod {
   delta_fcn                = DELTA_FUNCTION
   enable_logging           = ENABLE_LOGGING
   num_INS_cycles           = NUM_CYCLES
   needs_divfree_projection = FALSE
   rho_solid                = RHO
   PrintOutput {
   print_output          = TRUE                //default
  false
   output_interval       = 1                   //default 1
   output_drag           = TRUE                //default
  false
   output_power          = FALSE               //default
  false
   output_rig_transvel   = TRUE                //default
  false
   output_rig_rotvel     = TRUE                //default
  false
   output_com_coords     = TRUE                //default
  false
   output_moment_inertia = TRUE                //default
  false
   output_dirname        = "./Dump--Cylinder"  // default ./
  ConstraintIBMethodDump
   base_filename         = "Cylinder"      //default  "
  ImmersedStructure"
  }
}
```

`getFromInput` reads, besides `IBMethod`'s own keys: `needs_divfree_projection` (default false: project the spread correction to be divergence-free; 02 sets it), `rho_solid` (one value per structure; equal to the fluid's in every example), `rho_fluid`, `calculate_structure_linear_mom` and `calculate_structure_rotational_mom` (the body's own momenta, written to the log). The `PrintOutput` sub-block: `print_output`, `output_interval`, `output_drag`, `output_torque`, `output_power`, `output_rig_transvel`, `output_rig_rotvel`, `output_com_coords`, `output_moment_inertia`, `output_eulerian_mom`, `output_dirname`, `base_filename`, and `restart_output_dump_dir` for restarted runs. That is the whole list.

The lecture's slide 10 showed a different block: next slide.

THE LECTURE'S PRINTOUTPUT BLOCK, AND WHY NOTHING READ IT

```
// IBAMR Lecture 7 (2020), slide 10, "Input2d - print output", as
   shown
PrintOutput {

    print_output                = TRUE    //default false, if
  false nothing will be printed.
    output_interval             = 1       //default 1
    output_drag_kinetic_energy  = TRUE    //default false, if
  false it will not be computed.
    output_power                = FALSE    //default false, if
  false it will not be computed.
    output_rig_transvel         = FALSE    //default false
    output_rig_rotvel           = FALSE    //default false
    output_com_coords           = FALSE    //default false
    output_moment_inertia       = FALSE   //default false
    output_dirname              = "./cylinder_dump"
    base_filename               = "cylinder2d"
}
```

Two things are wrong with this block, and neither showed up in 2020:

- `output_drag_kinetic_energy` is not a key: `getFromInput` asks for `output_drag` (slide 14) and never looks for this one. An unknown key in an IBAMR input block is ignored without a word.
- the block sat at the *top level* of the upstream input, after `ConstraintIBKinematics`, not inside `ConstraintIBMethod`. The method reads only its own sub-block, so this one was never read at all: no `cylinder_dump` folder, no `cylinder2d_*` files.

The run therefore printed what the *inner* block (slide 14) asked for, into `Dump--Cylinder`, drag included. The upstream input still carries both blocks; ConstraintIB/01 removes the stray one and documents the real one. The general lesson is chapter 16's: a misplaced or misspelt key fails silently, so check the output folder, not the input, for what you asked for.

WHAT THE METHOD WRITES: THE FILES AND THEIR COLUMNS

Processor 0 opens one file per quantity in `output_dirname` at start-up, named `<base_filename>_<quantity>_struct_no_<k>` for structure k, and appends one tab-separated line every `output_interval` steps, time first (times $\rho_{\text{solid}}/\Delta t$ where a force is meant):

file	columns after the time
`_Drag_force_struct_no_k`	inertial force x, y, z (from the change of the points' velocity over the step); constraint force x, y, z (from the correction): their sum is the force on the fluid
`_Torque_struct_no_k`	torque, the same two parts
`_Power_spent_struct_no_k`	power of the constraint force
`_Trans_vel_struct_no_k`	rigid translational velocity x, y, z; velocity of the centre of mass due to the deformation x, y, z
`_Rot_vel_struct_no_k`	rigid angular velocity x, y, z; angular velocity of the deformation x, y, z
`_COM_coordinates_struct_no_k`	centre of mass x, y, z
`_MOI_struct_no_k`	the 3×3 moment-of-inertia tensor, row by row
`_EULERIAN_MOMENTUM.TXT`	total momentum of the fluid in the box x, y, z (one file for all structures)

Three columns in 2D as in 3D: the third is zero. The folder is created by the method; a file whose quantity is switched off is created empty. `np.loadtxt(name)` or MATLAB's `load(name)` reads any of them (slide 31); chapter 11's readers are not needed, since these are plain numbers.

WHAT HAPPENS EACH STEP, FROM THE SOURCE

`ConstraintIBMethod::postprocessSolveFluidEquations`, after the fluid solver has produced the new velocity:

1. `calculateCOMandMOIOfStructures`: the centre of mass and moment of inertia of each structure from its points (written if asked);
2. `interpolateFluidSolveVelocity`: the new fluid velocity interpolated to the points, $\mathbf{U}$;
3. `calculateKinematicsVelocity`: the accumulated angle is advanced, $\theta_n = \theta_{n-1} + \omega_{n-1}\Delta t$; the kinematics object's `setKinematicsVelocity` and `setShape` are called; for a self-translating body the momentum of the deformation velocity is computed, to be subtracted;
4. `calculateRigidTranslationalMomentum`, `calculateRigidRotationalMomentum`: for every free component, the rigid velocity is the *mean* of $\mathbf{U}$ over the structure's points (the momentum of the fluid in the body, per unit mass), and the angular velocity its moment about the centre of mass divided by the moment of inertia; locked components are zero;
5. `correctVelocityOnLagrangianMesh`: $\mathbf{U}_{\text{new}} = \mathbf{U}_{\text{def}} + \mathbf{U}_{\text{rigid}} - \mathbf{U}_{\text{def,COM}} + \boldsymbol{\Omega} \times \mathbf{R}$ at every point, and the correction $(\mathbf{U}_{\text{new}} - \mathbf{U})\times$ the volume per point;
6. `spreadCorrectedLagrangianVelocity`: the correction is spread to the grid, so the fluid in the body region now moves with the body; `applyProjection` if `needs_divfree_projection`;
7. `calculateDrag`, `Torque`, `EulerianMomentum`, `Power`, `StructureMomentum`: the outputs of slide 16;
8. then the IB integrator moves the points with the corrected velocity (`CONSTRAINT_VELOCITY`), or places them from the shape plus the rigid motion (`CONSTRAINT_POSITION`).

The "inertial force" of the drag file is $\rho_s(\mathbf{U}_{\text{new}} - \mathbf{U}_{\text{current}})\,V/\Delta t$ summed over the points, the "constraint force" $\rho_s\,\mathbf{U}_{\text{corr}}/\Delta t$ summed; the paper's section 7.1.4 is about why both are needed for a body whose velocity changes.

PRESCRIBED, FREE, AND IN BETWEEN: THE FLAGS

- All locked (`0,0,0` and `0,0,0`): the body moves exactly as the kinematics object says, and the force files tell you what that cost. The plate, the Stokes plate, the oscillating and the towed cylinder.
- Free rigid body (`1,1,0` and `0,0,1` in 2D) with zero kinematics velocity: the body moves with the momentum of the fluid in it, which the fluid got from a body force (the impulse of 02, gravity in the solid region for the falling sphere) or from the flow around it.
- Free swimmer: the same flags with a deformation velocity and shape: the eel, the 3D eel, the knifefish. The deformation's own momentum is removed, so a swimmer moves only by what it does to the fluid.
- Partly free: any mix, e.g. `1,0,0`: a body on rails.

Density. The formulation "assumes that the mass density of the fluid–structure system is uniform" (the paper, quoted by Lecture 6 for the falling sphere). `rho_solid` scales the forces; it does not give the body extra inertia. A heavier body in this method gets a body force equal to its excess weight, applied only inside the body: "we impose a gravitational body force that is present only in the solid region". Chapter 13's penalty method and IBFE with a density (Part IV) are the ways to a body that is really heavier.

Added mass comes for free: the fluid inside the body is part of the fluid, so an accelerating body drags its own volume of fluid along and the impulse of 02 is shared (slide 24).

CONSTRAINTIB/01: THE OSCILLATING CYLINDER

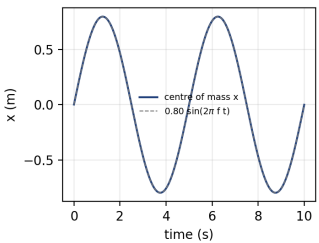
"This is a prescribed motion example from 7.1.4. 'In cases in which the body has a nonconstant specified velocity, contributions to the drag forces come from both inertial and constraint forces. In this example, we consider a rigid circular cylinder oscillating about a mean position in a quiescent fluid.'" (2020, slide 4)

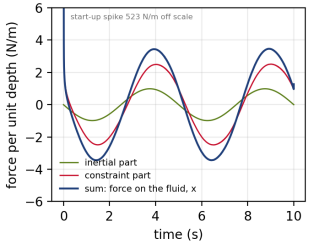
- a disc of diameter 1 m, $12{,}851$ points one finest cell apart (`Cylinder2d.m`: "The cylinder is filled in and ds=dx"; the lecture's "2dcylinder.vertex" is `cylinder2d.vertex`), velocity $U_\infty \cos 2\pi f t$ with $f = 0.2$ Hz and $U_\infty = 1$ m/s;
- $\mathrm{Re} = 100$ (`MU = 2.0*R/Re`), $\mathrm{KC} = U_\infty / fD = 5$: it travels 1.6 diameters each half period and meets its own wake;
- a 32×16 m periodic box, 64×32 cells on the coarsest level and four levels refined by 4, so the finest cells are $1/128$ m (128 across the disc); `DT_MAX = 0.0015`;
- "Be sure to include the header file when running simulations." (slide 6): `OscillatingCylinderKinematics.h`, and the `.cpp` in the CMake list.

```
// input2d: the lines slide 8 showed, as shipped (N is cells
  per 8 m)
R  = 0.5
Re = 100.0
MU = 2.0*R/Re
RHO = 1.0
MAX_LEVELS = 4         // maximum number of levels in locally
  refined grid
REF_RATIO  = 4         // refinement ratio between levels
REF_RATIO_FINEST = 4  // refinement ratio for the finest level.
N = 16                 // grid cells per 8 m on the coarsest
  level: ...
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N
L = 16
```

Slide 8 called `N` "coarsest grid spacing": it is a cell *count*, and in this input `CartesianGeometry` wrote the box as a literal `(63, 31)` that happens to equal $4N-1, 2N-1$; ConstraintIB/01 writes it with `N` so that changing `N` changes the grid. The run is two periods (10 s) instead of the upstream 130 s, and takes 11 minutes on 8 ranks of a laptop.

CONSTRAINTIB/01: WHAT IT DOES





Left: the centre of mass from `Cylinder_COM_coordinates_struct_no_0` against the integral of the prescribed velocity, $0.80 \sin 2\pi f t$. Right: the two parts of the force on the fluid from `Cylinder_Drag_force_struct_no_0`, and their sum.

Measured: the centre of mass reaches 0.7958 m at $t = 1.25$ s, which is $U_\infty / 2\pi f$, and never departs from $0.80 \sin 2\pi f t$ by more than 10^{-6} m. The force: next slide.

ConstraintIB/01 with IBAMR 0.19.0, 8 ranks; `tools/ch14_figures.py` reads the method's text outputs.

CONSTRAINTIB/01: READING THE FORCE

- In the second period the force on the fluid swings between -3.44 and $+3.46$ N/m. Its Fourier component in phase with the acceleration is 3.36 N/m; the inviscid estimate for a disc of the fluid's density, the displaced fluid plus an equal added mass, is $2\rho\pi R^2 a_{\max} = 1.97$ N/m, so the fluid at $\mathrm{Re} = 100$ and $\mathrm{KC} = 5$ resists the acceleration 1.7 times more than ideal flow would: the wake the disc runs back into is part of what it must accelerate.
- The component in phase with the velocity is 0.91 N/m, a drag coefficient $2F/\rho U_\infty^2 D$ of 1.8; the drag on the cylinder is minus the force on the fluid.
- The first period's force has the same shape except for its first step, a spike of 523 N/m that the plot's scale leaves out: the velocity jumps from 0 to U_∞ at $t = 0$ and the method supplies that impulse in one step. A ramp (`U_infinity` times $\min(t/t_r, 1)$ in the kinematics class) removes it; the upstream example does not bother, since it runs 26 periods.
- The two parts the file keeps separate (slide 16): the inertial part follows the acceleration of the body's own fluid, the constraint part is everything the fluid outside did not already provide; the paper's section 7.1.4 is about why a body with a changing velocity needs both.

The lecture's picture (slide 4) was VisIt's vorticity at $t = 72$ s with the disc's points over it: a pair of vortices either side of the disc, blue and red, left behind at each reversal. Chapter 26's recipe makes it from `viz_cylinder2d`.

CONSTRAINTIB/02: THE IMPULSIVELY STARTED CYLINDER

"This example appears in the paper in section 7.2.2. 'We take the case of impulsively started cylinder in a quiescent flow to demonstrate the momentum-conservation properties of our method. A cylinder of diameter $D = 1$ m is placed at the center of a periodic domain of size 32 m x 16 m at time $t = 0$. An impulse of the form $I(t) = 400 \exp(-400t)$ N/m^3 is applied in the x direction within the body, at its initial position, thereby impulsively starting its motion.' This is an example of rigid body motion." (2020, slide 13)

"Cylinder2d.vertex. As before, contains a list of x,y coordinates of the vertices. Spacing is dx=ds." (slide 14): 795 points $1/32$ m apart, three levels, from the same generator as 01 with 1024 cells per 32 m.

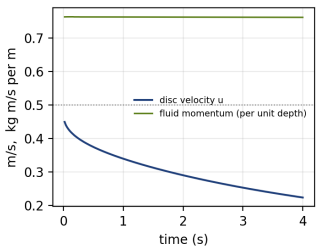
```
// input2d (ConstraintIB/02): the impulse is a body force on the
  fluid
ForcingFunction {
    XCOM = 0.0
    YCOM = 0.0
    RADIUS = R
    F      = RHO/DT_MAX
    function_0 = "(X0 - XCOM)^2 + (X1-YCOM)^2  < RADIUS^2
                    ? 400*exp(-400*t) : 0"        // one line in
  the file
    function_1 = "0.0"
}
ConstraintIBKinematics {
     Cylinder2d {
     structure_names                  = "cylinder2d"
     structure_levels                 =  MAX_LEVELS - 1
     calculate_translational_momentum = 1,1,0
     calculate_rotational_momentum    = 0,0,1
     lag_position_update_method       = "CONSTRAINT_VELOCITY"
     tagged_pt_identifier             = MAX_LEVELS - 1, 0
     kinematics_velocity_function_0   = "0.0"
     kinematics_velocity_function_1   = "0.0"
    }
}
```

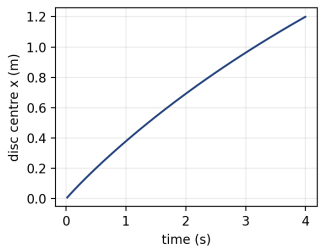
CONSTRAINTIB/02 (2): WHAT TO NOTICE, AND SLIDE 15 CORRECTED

- the force lives in the input: a muParser expression with a ternary, non-zero inside the disc only, registered by the `ForcingFunction` hook of `main.cpp` (chapter 7); its time integral is 1 N s/m^3, so the impulse is the disc's area, $\pi/4$ per metre of depth; `F = RHO/DT_MAX` is defined and unused;
- the body is free in x, y and rotation; its kinematics velocity is zero ("0.0" for both components; a missing one gives a warning and zero);
- `needs_divfree_projection = TRUE` and `output_eulerian_mom = TRUE` in the method's block: the correction is projected, and the fluid's total momentum is written to `CylinderDiv_EULERIAN_MOMENTUM.TXT`;
- `Re = 200` sets `MU = 2.0*R/Re`; the run is 4 s instead of "10000" (run until stopped), two and a half minutes on 4 ranks.

Slide 15 corrected. "RigidBodyKinematics.cpp. This file is used to determine the rigid body motion of the cylinder. An applied force that decays over time is applied to the cylinder. The center of mass is calculated as well as the translational and rotational momentum. This is used to update the position of the immersed boundary. Remember to include RigidBodyKinematics.h" The class does none of the middle three things: it evaluates the two `kinematics_velocity_function` expressions, nothing more; the force is the input's `ForcingFunction`, applied by the fluid solver, and the centre of mass and momenta are the method's, by the `calculate_*` flags. The header must indeed be included. Slide 17's `function_1 = "0.0‘‘` ended in a curly quote, which the input parser rejects; the file has straight quotes (chapter 16).

CONSTRAINTIB/02: WHAT IT DOES





Left: the disc's velocity (`CylinderDiv_Trans_vel_struct_no_0`) and the fluid's total momentum (`CylinderDiv_EULERIAN_MOMENTUM.TXT`). Right: the disc's position.

Measured: the disc's velocity peaks at 0.449 m/s at $t = 0.02$ s, when the impulse is spent, against 0.5 for an ideal disc of the fluid's density taking half the impulse; it is 0.42 at 0.1 s, 0.34 at 1 s and 0.22 at 4 s, by which time the disc has moved 1.20 m. The momentum: next slide.

ConstraintIB/02 with IBAMR 0.19.0, 4 ranks; `tools/ch14_figures.py`.

CONSTRAINTIB/02: READING THE MOMENTUM

- The fluid's total momentum (`CylinderDiv_EULERIAN_MOMENTUM.TXT`) is 0.764 at 0.1 s and 0.762 at 4 s: conserved to 0.2 per cent over $2{,}000$ steps. The value is what the impulse delivered: 795 forcing cells of $1/32$ m give 0.776 (the disc's area is 0.785; the ternary counts cell centres).
- Why the disc's velocity is half of the impulse per unit mass: the impulse is delivered to the fluid in the disc region; to move, the disc must also move the fluid around it, and for a cylinder the added mass equals the displaced mass, so a body of the fluid's density takes half the momentum and leaves half in the fluid. The measured 0.449 against 0.5 is the viscous boundary layer and the grid's rendering of the disc's edge.
- The total stays put because the constraint force only moves momentum between body and fluid, it makes none, and a periodic box has no walls to absorb it. Then viscosity slows the disc at $\mathrm{Re} = 200$: 0.42 at 0.1 s, 0.34 at 1 s, 0.22 at 4 s.
- The angular velocity (`CylinderDiv_Rot_vel_struct_no_0`) stays below 0.006 s^{-1}: nothing in a symmetric problem turns the disc, and the method does not invent a torque.

CONSTRAINTIB/03: THE SWIMMING EEL, AS THE LECTURE INTRODUCED IT

“ConstraintIB – eel2d. In this example, the body deforms in a prescribed manner but is propelled through the water due to rigid body motion. The center of mass is calculated at each time as well as the translational and rotational momentum. This is used to update the position.” (2020, slide 19, with the VisIt picture of slide 29)

“eel2d_straightswimmer. This is a MATLAB code that generates the vertices for the eel. Eel2d.vertex is the output file.” (slide 20)

“IBEELKinematics.cpp. The default just allows for straight swimming of the eel. The deformation of the body is prescribed. The propulsion is based on rigid body kinematics. There are other functions for chasing a target / food, swimming in a circle, etc. This makes the code a little harder to understand. `if (d_bodyIsManeuvering)` – eel is maneuvering (turned off now). In such cases, the maneuver axis is used to transform the body. Deformation velocity functions are read in from the input file. Center of mass is calculated at each time.” (slide 21)

Slides 22 and 23 (“Input2d – Constraint IB Method”, “Input2d – body shape equation”) were screenshots of the two input blocks, retyped on the next slide.

CONSTRAINTIB/03 (2): THE INPUT BLOCK

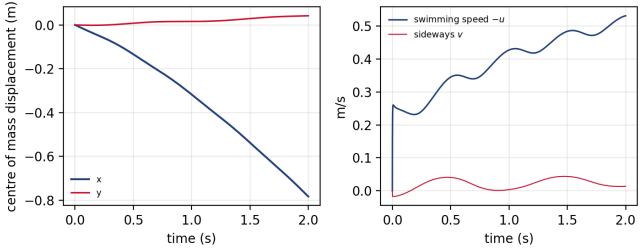
```
// input2d (ConstraintIB/03): the eel's sub-block; the three long expressions are one line each in the file
eel2d {
     structure_names                  = "eel2d"
     structure_levels                 =  MAX_LEVELS - 1
     calculate_translational_momentum = 1,1,0
     calculate_rotational_momentum    = 0,0,1
     lag_position_update_method       = "CONSTRAINT_POSITION"
     tagged_pt_identifier             = MAX_LEVELS - 1, 0  // level, relative idx of lag point
     initial_angle_body_axis_0         = 0.0                       //default 0.0
     body_shape_equation               = "0.125* ( (X_0 + 0.03125)/1.03125 ) * sin( 2*PI*X_0 - (0.785/0.125)*T )"
     deformation_velocity_function_0   = "( -0.785* ( (X_0 + 0.03125)/1.03125 )
                                          *cos( 2*PI*X_0 - (0.785/0.125)*T ) )*N_0"
     deformation_velocity_function_1   = "( -0.785*  ( (X_0 + 0.03125)/1.03125 )
                                          *cos( 2*PI*X_0 - (0.785/0.125)*T ) )*N_1"
     body_is_maneuvering                = FALSE       //default false. If FALSE then nothing below has any effect.
     maneuvering_axis_equation          = "sqrt(1.3^2 -(X_0 - 0.5)^2) - 1.2"     //"0.1*sin(PI*X_0)"
     maneuvering_axis_is_changing_shape  = TRUE  // should be true.
     food_location_in_domain_0         =  1.0
     food_location_in_domain_1         = -3.3
}
```

The shape in the body frame, $y = 0.125\,\frac{x+0.03125}{1.03125}\,\sin(2\pi x - 6.28\,t)$ for x from the nose (0) to the tail (1): one wavelength on the body, travelling tailward at one length per second, amplitude from 0.004 at the nose to 0.125 at the tail. The velocity functions are its time derivative, applied along the body normal (`N_0`, `N_1`; slide 13). The amplitude and frequency appear in all three expressions and must be changed together.

CONSTRAINTIB/03 (3): WHAT TO NOTICE

- `CONSTRAINT_POSITION`: each step the class *sets* the shape (`setShape`) and the method adds the rigid translation and rotation it computes, so the body never drifts from its prescribed form; with `CONSTRAINT_VELOCITY` the integrated velocity would slowly deform it;
- the flags unlock x, y and rotation: the eel yaws as well as swims, and the `Rot_vel` file records it; the deformation's own momentum (the wave as written is not momentum-free) is measured each step and subtracted, so only the fluid can propel the eel;
- $2{,}932$ points filling the body, one finest cell apart, from `eel2d_straightswimmer.m`: a rounded head 0.04 wide and a linear taper to the tail; nose at $x = -0.5$, tail at 0.5; the Python port writes the same file;
- $\mathrm{Re} = 5609$ on the tail's lateral speed 0.785 (`MU = 0.785/Re`); an 8×4 m periodic box with six body lengths of room ahead; finest cells $1/256$ m (ten across the head); `DT_MAX = 0.0001`, $10{,}000$ steps per beat;
- `main.cpp` also carries an `IBHydrodynamicForceEvaluator`: the force on the eel from a control volume (`InitHydroForceBox_0`, 2×1.4 m) that moves with it, written to `Drag_CV_strct_id_0` and `Torque_CV_strct_id_0` (chapter 11's other way to a force; Nangia et al. 2017);
- two beats (2 s) instead of the upstream 10 s, 12 minutes on 8 ranks; the upstream restart dumps every 150 steps are switched off.

CONSTRAINTIB/03: WHAT IT DOES



Left: the centre of mass (`Eel2d_COM_coordinates_struct_no_0`). Right: the swimming speed and the sideways velocity (`Eel2d_Trans_vel_struct_no_0`).

The numbers: next slide.

ConstraintIB/03 with IBAMR 0.19.0, 8 ranks; `tools/ch14_figures.py`.

CONSTRAINTIB/03: READING THE SWIM

Measured: the centre of mass moves -0.32 m in the first beat and -0.47 m in the second, -0.78 m in all with 0.04 m sideways: the eel is still accelerating, from 0.32 to 0.47 lengths per beat (the speed jumps to 0.26 m/s in the first step, because the wave starts at full amplitude at $t = 0$, and climbs from there with a dip at every half beat). The lecture's picture at $t = 1$ s, the body between $x = -0.8$ and 0.25, is the first beat's 0.32. The yaw rate reaches 0.47 s^{-1}; the deformation's centre-of-mass velocity, which the method removes each step, reaches 0.07 m/s.

What to take from it: a free swimmer in the constraint method moves only by what its prescribed wave does to the fluid, so the speed is an outcome, not an input; two beats are not enough for a steady speed (the upstream 10 s run is), and the eel yaws a little as it goes, which `calculate_rotational_momentum = 0,0,0` would forbid.

The lecture's picture (slide 19, VisIt, $t = 1$ s, cycle 10000): the body between $x = -0.8$ and 0.25, having started between -0.5 and 0.5, with the boundary layers of opposite sign along its two sides and a vortex pair shed from the tail trailing behind at $x \approx 0.4$ to 0.5. The wave travels tailward, so the eel swims toward $-x$. Chapter 26 reproduces it from `viz_eel2d_Str`.

READING THE OUTPUTS

```python
# Python: the disc's velocity after the impulse (ConstraintIB/02)
import numpy as np
v = np.loadtxt("Cylinder_Dump/CylinderDiv_Trans_vel_struct_no_0")
t, u = v[:, 0], v[:, 1]            # time, rigid velocity x
print(u.max(), t[u.argmax()])      # the peak and when
p = np.loadtxt("Cylinder_Dump/CylinderDiv_EULERIAN_MOMENTUM.TXT")
print(p[-1, 1])                    # the fluid's x momentum at
  the end
```

```octave
% MATLAB/Octave: the force on the oscillating cylinder (
  ConstraintIB/01)
d = load('Dump--Cylinder/Cylinder_Drag_force_struct_no_0');
t = d(:,1); F = d(:,2) + d(:,5);   % inertial + constraint, x
plot(t, -F); xlabel('t (s)'); ylabel('drag on the cylinder (N/m)')
  ;
```

The files are plain tab-separated numbers with no header, so `np.loadtxt` and `load` read them whole; the column meanings are on slide 16. Two cautions:

- a file is written every `output_interval` steps, and the position file is written at the *start* of the step (`d_FuRMoRP_current_time`) while the velocity and force files carry the end (`new_time`): their first lines differ by one step;
- a quantity that was switched off leaves an empty file, which `loadtxt` refuses; check `PrintOutput` rather than the folder.

`tools/ch14_figures.py` in the tutorial repository does this for the three examples and prints the numbers quoted on their slides; it is twenty lines per example.

RUNNING THE THREE ON PUMA, AND WHAT THEY COST

```
# from the example folder, after enable.sh (chapter 1)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 4
sbatch job.slurm          # 4 tasks; edit --account; the outputs: viz_*/ for VisIt, the text files
  in Dump--Cylinder/, Cylinder_Dump/ or Eel2dStr/
```

example	steps	finest cells	levels	ranks	wall time
01 oscillating cylinder, two periods	$6{,}667$	$1/128$ m	4	8	11 min
02 impulsively started cylinder, 4 s	$2{,}000$	$1/32$ m	3	4	2.5 min
03 eel, two beats	$20{,}000$	$1/256$ m	3	8	12 min

Each folder's `job.slurm` is chapter 1's with the job name and a time limit chosen from these runs; `CMakeLists.txt` lists the program and the kinematics class. The times are from a laptop under WSL (chapter 2), the three running at once with `mpirun --bind-to none`. The upstream lengths (130 s, "10000" s and 10 s) are 13, 2500 and 5 times longer: those are Puma jobs, and the reason for the lecture's warning about files.

WHAT CAN GO WRONG (1): THE MESSAGES

what you see	why	fix
`Structure cylinder2d does not exist on level 2`	`structure_levels` in the kinematics block and `level_number` in `IBStandardInitializer` disagree, or the name is misspelt	make the three names and the two levels agree
`Update methods supported are CONSTRAINT_VELOCITY, CONSTRAINT_POSITION AND CONSTRAINT_EXPT_POSITION`	`lag_position_update_method` is anything else	use one of the three
`Could not tag Lagrangian point for this structure`	`tagged_pt_identifier`'s level is not one of the structure's	level first, then the relative index
`frequency of cylinder does not exist in the InputDatabase`	the subclass requires its keys	add `frequency`, `U_infinity`
`Total levels > 1, OscillatingCylinder should be places on only one level`	the structure spans levels	put it on one level
`no function corresponding to key kinematics_velocity_function_1`	a warning, not a stop: the component is set to `"0.0"`	add the key if you meant something else

The messages are the sources' (`ConstraintIBKinematics.cpp`, `OscillatingCylinderKinematics.cpp`, `RigidBodyKinematics.cpp`); "places" is the source's spelling. Chapter 16 has the general ones (CFL, parse errors, Puma limits).

WHAT CAN GO WRONG (2): THE SILENT ONES

what you see	why	fix
nothing is printed, or the wrong things	`PrintOutput` must be *inside* `ConstraintIBMethod`; unknown keys there are ignored without a word (the 2020 slide 10)	the list of slide 14
a parse error at a quote	a curly quote pasted from a slide (`"0.0“`)	straight quotes (chapter 16)
the disk fills	restart every 150 steps, viz every 40, 100,000 steps	raise the intervals; the examples' READMEs
the body drifts from its shape	`CONSTRAINT_VELOCITY` with a deformation integrates errors	`CONSTRAINT_POSITION`, as the eel
the body does not move although it is free	the deformation carries momentum, which the method removes; or the flags are `0,0,0`	check `calculate_*`; give the fluid a push (a body force) or a wave that pushes fluid
a heavier body falls no faster	`rho_solid` does not add inertia	a gravity body force in the body's region (the falling sphere), or chapter 13

CONSTRAINT IB AMONG IBAMR'S RIGID-BODY TOOLS

- `ConstraintIBMethod` (this chapter): a region of fluid made to move as a rigid or prescribed-deformation body by a velocity correction after each solve; free motion from the momentum in the body; 2D and 3D; `examples/ConstraintIB`.
- `CIBMethod` (chapter 15 tours its `examples/CIB`): the constraint written as a saddle-point problem with a mobility solver, for Stokes and low-Reynolds-number suspensions of many rigid bodies; the lecture: "Some examples are also similar to the CIB ones."
- `IBFEDirectForcingKinematics`: the same idea for an IBFE mesh (Part IV), with the body a finite-element volume.
- `BrinkmanPenalizationRigidBodyDynamics`: a level-set body made rigid by a Brinkman penalty term in the momentum equation, with the body's dynamics integrated; the level-set examples.
- `PenaltyIBMethod` (chapter 13): mass on a copy of a point; not rigid, but heavy.
- Target points (chapter 9a): a rigid body of the standard method, held by tethers; the simplest, and the stiffest.

> Prescribed motion of a rigid shape through a fluid, or a body whose deformation you know: constraint IB. A body whose deformation the fluid decides: IBFE. Many rigid particles in Stokes flow: CIB.

HOMEWORK

1. ConstraintIB/01: do the lecture's exercise (slide 7, "Alter the kinematics to make the cylinder go up and down"): write `[0][1][k]` instead of `[0][0][k]` and compare the y force file with the x one of the shipped run. Then tow it (`U_infinity = 0`, `prescribed_trans_velocity = 1`) and read the drag coefficient $C_D = 2F/(\rho U^2 D)$ from the force file after it settles (chapter 11).
2. ConstraintIB/02: push the disc off centre (`XCOM = 0.2`) so that the impulse has a moment; read the angular velocity file. Then repeat with `needs_divfree_projection = FALSE` and say what changed in the momentum file.
3. ConstraintIB/03: halve the amplitude ($0.125 \to 0.0625$ in all three expressions) and run two beats; compare the swimming speed. Then switch `body_is_maneuvering = TRUE` and watch where the eel goes.
4. Write a kinematics class of your own: a disc that rolls (prescribed angular velocity, zero translation) by filling `d_current_kinematics_vel` with $\mathbf{\Omega} \times \mathbf{R}$ about the centre of mass, which `setKinematicsVelocity` receives. Start from `OscillatingCylinderKinematics`.
5. Add a second structure: two discs, each with its own kinematics object and sub-block, `num_structures = 2`; the force files get a `_struct_no_1`.

RULES OF THUMB

- `ConstraintIBMethod` + one kinematics object per structure + two input blocks; register, then `initializeHierarchyOperatorsandData()`.
- A kinematics class sets a velocity per point per component; the method adds the rigid motion for the free components (`calculate_*` flags) and removes the deformation's own momentum.
- One level per structure; names and levels must agree in three places.
- `PrintOutput` inside the method's block; the keys of slide 14; the files of slide 16; `np.loadtxt` reads them.
- Body density is the fluid's; a heavier body gets a body force in its region.
- Prescribed deformation: `CONSTRAINT_POSITION`. Rigid: `CONSTRAINT_VELOCITY`.
- Fill the body with points one finest cell apart (`Cylinder2d.m`: "ds = dx").
- Shorten the upstream runs and dump intervals before the first run.

SUMMARY

1. The constraint IB method solves the fluid everywhere and then corrects the velocity inside a body to the rigid or prescribed-deformation motion the body should have; the correction's cost is the hydrodynamic force, written every step.
2. A program needs `ConstraintIBMethod`, a `ConstraintIBKinematics` subclass with four functions, and two input blocks: the structure's (names, levels, the free-component flags, the update method, a tagged point, the class's own keys) and the method's with `PrintOutput`.
3. Locked flags give prescribed motion (01); free flags with zero kinematics give a rigid body moved by the fluid's momentum (02); free flags with a deformation give a swimmer (03).
4. The outputs are plain text: drag, torque, power, velocities, centre of mass, moments of inertia, the fluid's momentum; four lines of Python or MATLAB read them.
5. The three examples run in minutes to an hour on a laptop and reproduce the lecture's pictures; the 2020 slides' three slips (a key that does not exist, a file name, what `RigidBodyKinematics` does) are corrected here.

Chapter 15 tours the rest of IBAMR's examples, ConstraintIB's nine among them; chapter 16 collects what can go wrong.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The 2020 lecture showed files; this chapter says what the method does with them (slides 5, 17, 18), from the source, and runs the three examples with measured results.
- The input keys are the verified list: slide 10's `output_drag_kinetic_energy` does not exist and its block was never read; slide 8's "coarsest grid spacing" is a cell count; slide 5's `2dcylinder.vertex` is `cylinder2d.vertex`; slide 15's account of `RigidBodyKinematics.cpp` is corrected; slide 17's curly quote is straightened.
- The examples are shipped with the tutorial's CMake build, shortened runs, Python ports of the generators checked against Octave and against the upstream files, READMEs with the output formats, and a figure script.
- Lecture 6's six slides on the ConstraintIB set are merged in (the table of slide 7), so the chapter stands alone; the CIB set is left to chapter 15.

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recordings of IBAMR Lecture 7 (*Constraint IB*, two files, with and without sound, 2020-09-09) and of Lecture 6 (*Boyce's examples*) exist in the course's Drive folder and are listed in the front matter's video list (chapter 0.1). They were taught from the slides quoted here, and nothing in them is reproduced in this chapter.

REFERENCES

- A. P. S. Bhalla, R. Bale, B. E. Griffith and N. A. Patankar, *A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies*, Journal of Computational Physics 250 (2013) 446–476.
- N. Nangia, H. Johansen, N. A. Patankar and A. P. S. Bhalla, *A moving control volume approach to computing hydrodynamic forces and torques on immersed bodies*, Journal of Computational Physics 347 (2017) 437–462 (the force evaluator of ConstraintIB/03).
- IBAMR 0.19.0: `include/ibamr/ConstraintIBMethod.h`, `src/IB/ConstraintIBMethod.cpp`, `include/ibamr/ConstraintIBKinematics.h`, `src/IB/ConstraintIBKinematics.cpp`, `examples/ConstraintIB/` (nine examples). https://github.com/IBAMR/IBAMR
- The examples of this series: ConstraintIB/01 oscillating cylinder, 02 impulsively started cylinder, 03 swimming eel (each with `main.cpp`, a kinematics class, `input2d`, a vertex file and its MATLAB and Python generators, `CMakeLists.txt`, `job.slurm`, README); `tools/ch14_figures.py`.
- Sources: *IBAMR Lecture 7: Looking at ConstraintIB examples of prescribed and rigid body motion* (2020, 23 slides) and *IBAMR Lecture 6: What do Boyce's examples do* (2020), slides 17–24.